

Adding a Product Pair

Every sellable product is a matched pair — the POD listing and its in-house TWC twin. Work top to bottom. The order is load-bearing in three places, and each one is flagged.

REV 2026-09-08
OWNER Larry Williams
TARGET <10 min
EXCL. ARTWORK

JOB _____ DATE _____ OPERATOR _____

A Build the POD half

POD PLATFORM

☐ 01 **Build the product with the full colour and size run**

Create it in the POD app. Choose every colourway and size you intend to sell — not a subset you plan to extend later. Generate mockups.

Why the full run: the in-house twin inherits this grid. If the POD half is narrower, the flip guard blocks the capacity valve later, exactly when you need it.

SCREENSHOT 1

The POD app's variant grid with every colour and size selected.
Frame: the full variant matrix. Crop out the browser chrome.

☐ 02 **Set retail price here — not in Shopify**

Price to the standard ladder inside the POD app.

ADULT COMFORT COLORS TEE	S-XL	2XL	3XL	4XL
Every colourway	\$22.00	\$24.00	\$25.00	\$26.00

Order matters. The POD app is upstream of Shopify. A price set in Shopify is overwritten on the next sync; a price set in the POD app survives. Confirmed the hard way on 2026-09-08.

SCREENSHOT 2

The POD app's pricing screen with the ladder entered.
Frame: the price column beside the size column, so both read together.

☐ 03 **Publish to Shopify**

Publish from the POD app. If you edit anything after this — price especially — you must *republish*, not just save. A price change alone does not sync.

B Attribute the POD half

SHOPIFY ADMIN

☐ 04 **Set Vendor to the exact partner org string**

Product organization card, right-hand column. Type it exactly — the match is literal.

ORG KEY	VENDOR STRING
gcs	Germantown Christian School
gcs-athletics	GCS Athletics
revive	Revive Church
fhcog	Free Holiness Church of God
hoh	Highway of Holiness
community-christian	Community Christian
men-of-faith	The Men of Faith
alt-west	ALT-WEST
steeple-stitch	Steeple & Stitch Co.

Vendor is the one load-bearing field. Every storefront collection is a `VENDOR EQUALS` smart rule, so vendor *is* the collection assignment. Printify writes `Printify`; Ninja POD leaves `Steeple & Stitch Co.` Neither matches an org, so neither lands anywhere.

SCREENSHOT 3

Shopify product page → **Product organization** card, corrected.

Frame: the whole card – Type, Vendor, Collections, Tags. On a correct GCS Athletics product all four read `Apparel` / `GCS Athletics` / `GCS Athletics` / `GCS Athletics`.

☐ 05 Confirm the vendor took

Check the product carries no `needs-vendor-fix` tag. The Flow workflow of that name tags anything created with a vendor matching no known org.

SCREENSHOT 4

Products list filtered on the `needs-vendor-fix` tag, returning nothing.

Frame: the filter chip and the empty result together – the pair is the proof. Save this as a Products view if you haven't.

☐ 06 Note the product ID

It's the last segment of the admin URL. The clone command takes it.

```
https://admin.shopify.com/store/<store>/products/15262932533616
```

c Clone the in-house half

TERMINAL · POD2TWC

☐ 07 Dry run the clone and read the plan

```
P=~/.venvs/pod2twc/bin/python
$P ~/Dev/pod2twc/pod2twc.py clone <product-id> --org gcs-athletics --cost 14.00
```

`--cost` is your **fully loaded** in-house cost, labor included. It is expected to sit *above* the POD basis. That is deliberate and must not be "corrected".

FLAG	WHEN TO REACH FOR IT
<code>--pod-cost N</code>	The POD app pushed cost = retail (Printify does this). Supply the true cost.
<code>--price N</code>	Override retail across all variants.
<code>--keep-images</code>	Copy the POD mockups instead of starting empty.
<code>--no-flip</code>	Leave the clone DRAFT and the POD half untouched.

SCREENSHOT 5

Terminal showing the dry-run plan.

Frame: enough of the output to show the inventory-location lines – that's the part worth reading before committing.

☐ 08 Commit it

```
$P ~/Dev/pod2twc/pod2twc.py clone <product-id> --org gcs-athletics --cost 14.00 --commit
```

The clone lands ACTIVE, the POD half goes DRAFT, and the two are linked by metafield. This replaces the entire copy/paste pass.

The step that actually matters happens inside this command: a plain Shopify duplicate stays stocked at the POD fulfillment location, so an order on the in-house listing routes straight back to the printer — POD cost paid on a garment you printed yourself. The script turns tracking off *first*, detaches every POD location, then stocks the house and pickup locations.

☐ D Finish the in-house half

SHOPIFY ADMIN • TERMINAL

☐ 09 Upload your own mockups

The clone starts with no images by design — POD mockups show the printer's render, not your artwork.

☐ 10 Record any supplier coverage gaps

```
$P ~/Dev/pod2twc/pod2twc.py except <ref> # show state  
$P ~/Dev/pod2twc/pod2twc.py except <ref> --add "White / S" --commit
```

Some gaps are permanent — the supplier simply doesn't make that combination. Record them so the guard stays quiet about those and loud about everything else.

Don't train yourself to ignore the guard. Forcing past it every time is how a real gap gets shipped.

☐ 11 Eyeball the metafields on the product page

All six definitions are pinned, so they sit directly on the product page rather than behind “Show all”.

#	FIELD	SHOULD READ
1	Church Store	the partner org's collection
3	Church Store Name	the org's display name
4	Church Store URL	the org's collection URL
5	POD Source Product	the POD half of this pair
6	POD Basis Cost	the POD charge — the payout basis
7	Pair Coverage Exceptions	blank, or accepted gaps

POD Basis Cost is not your cost. It's what the printer charges, and it's what the partner org's payout is calculated against — whichever half fulfills. Your loaded in-house cost lives in Cost per item and is deliberately higher.

SCREENSHOT 6

Product page metafields, all six pinned fields populated.

Frame: the pinned block top to bottom. Use a fully finished pair as the example.

E Verify

TERMINAL

☐ 12 Audit the pair

```
$P ~/Dev/pod2twc/pod2twc.py audit
```

The new pair should report no gaps. Fix anything it flags now — a gap found later is a gap found while you're drowning.

SCREENSHOT 7

Terminal showing a clean audit.

Frame: the summary line plus the new pair's row.

☐ 13 Confirm the pair is live the right way round

In-house twin **ACTIVE**, POD half **DRAFT**. Retail identical on both halves — if they disagree, the twin's price is the correct one, and the fix goes in at the POD app followed by a republish.

The capacity valve — flipping to POD

Set the trigger in advance, not while you're behind. Proposed default: flip when the oldest unfulfilled order passes **5 business days**, or open orders exceed **10**, whichever comes first.

```
$P ~/Dev/pod2twc/pod2twc.py flip <ref> --to pod          # dry run
$P ~/Dev/pod2twc/pod2twc.py flip <ref> --to pod --commit
```

THE GUARD SAYS

YOU DO

A gap the supplier genuinely can't fill

`except ... --add` it, then flip

A gap you can fix

Add the variant in the POD app, republish, flip

A gap you'll accept losing for now

`--force`, and write down that you did

Flipping back is `flip <ref> --to twc --commit`. Nothing is deleted in either direction.

Weekly, not once

Run `audit` and `metafields` every week. They are the readiness check on the valve — the moment you need it is the moment you won't have time to check it.

Steeple & Stitch Co. · shop traveler for the dual-SKU product model

Full detail: `claude/ops/11-pod-to-inhouse-clone.md` · `claude/ops/14-new-product-sop.md` · `~/Dev/pod2twc/README.md`

Progress is saved in this browser only. “Start new job” clears it for the next product.